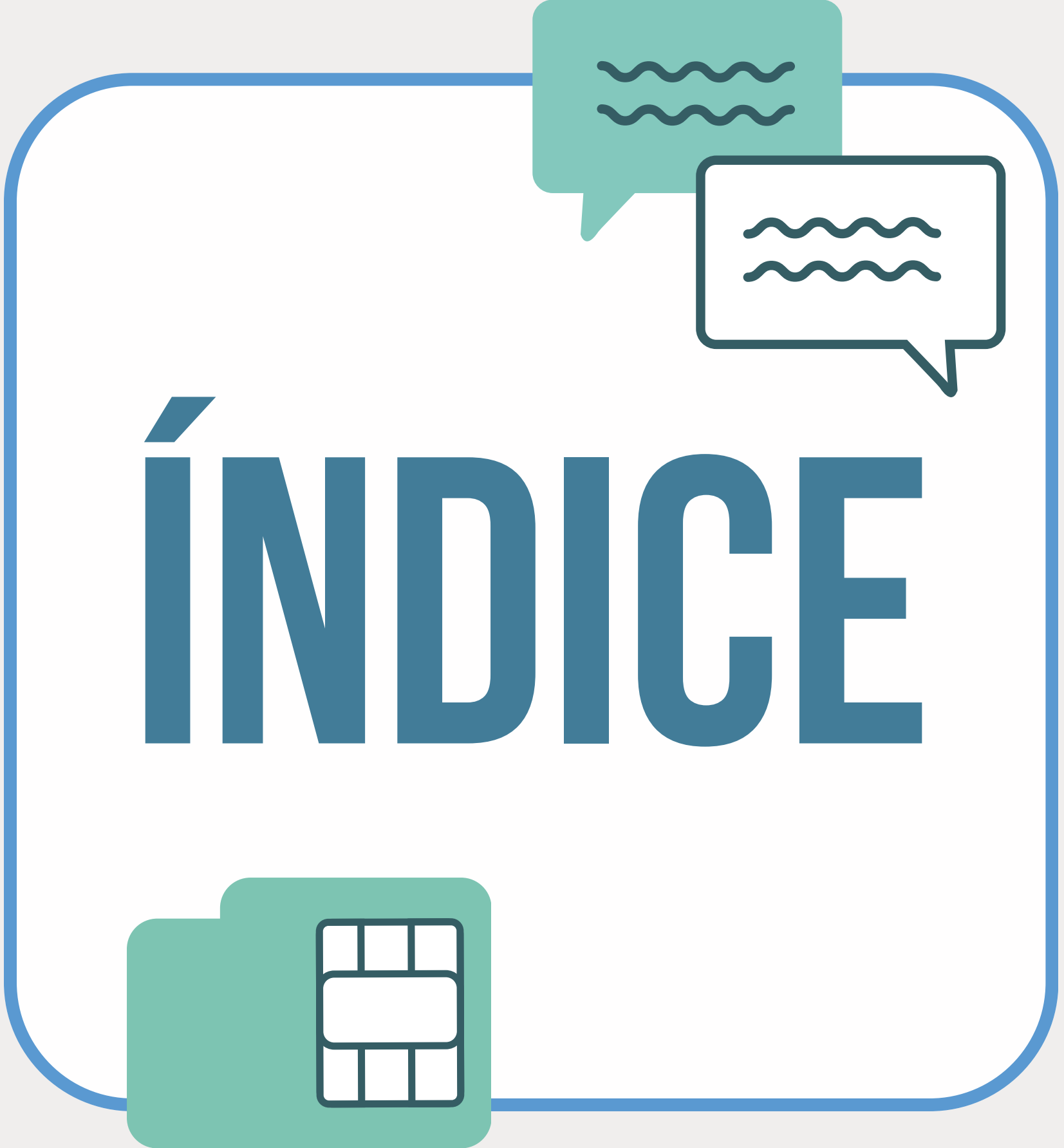




NOSQL

Roberto Ibaceta Cisternas



ÍNDICE

01. Contexto & Origen de NoSQL

02. Que no es NoSQL?

03. ACID vs BASE

04. Cuando No usar NoSQL

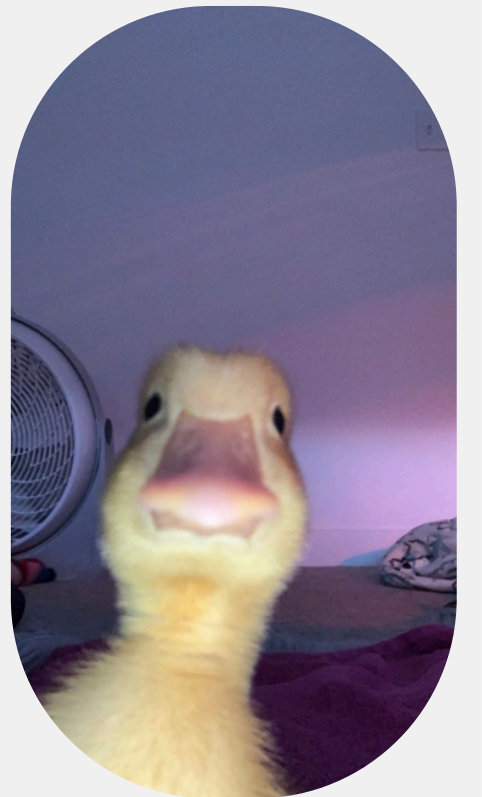
05. Familias NoSQL

06. Grafo

07. Demo

Características Principales

- **Almacena información no estructurada.**
- **Pensada para almacenar grandes volúmenes de datos.**
- **Sacrifica consistencia para ofrecer eficiencia.**
- **No hay un lenguaje común para administrar las bds.**
- **Orientadas a sistemas de gestión (OLAP)**



Ventajas

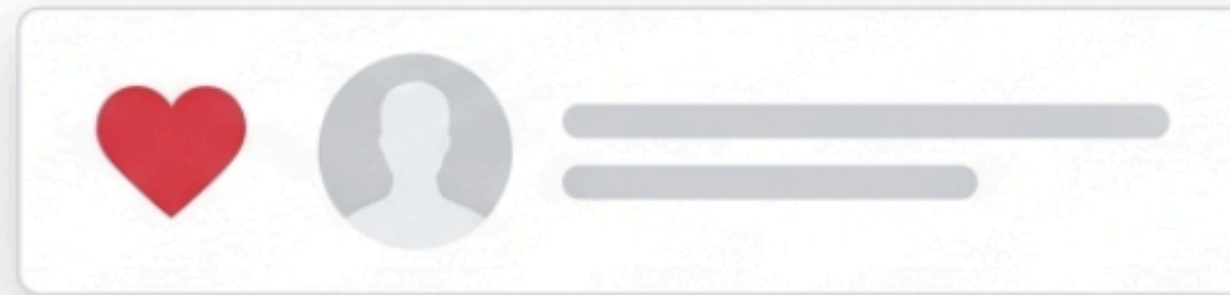
- **Almacenamiento flexible de los datos:**
- **Alta disponibilidad y rendimiento**
- **Permite manejar masivos volúmenes de datos.**
- **Permite escalar horizontalmente.**
- **Simplifica modificar el esquema de los datos.**



Desventajas

- **No hay un lenguaje estándar de administración de la base de datos.**
- **No asegura la consistencia de los datos.**
- **Mayor consumo de almacenamiento.**

3.000.000.000 Usuarios

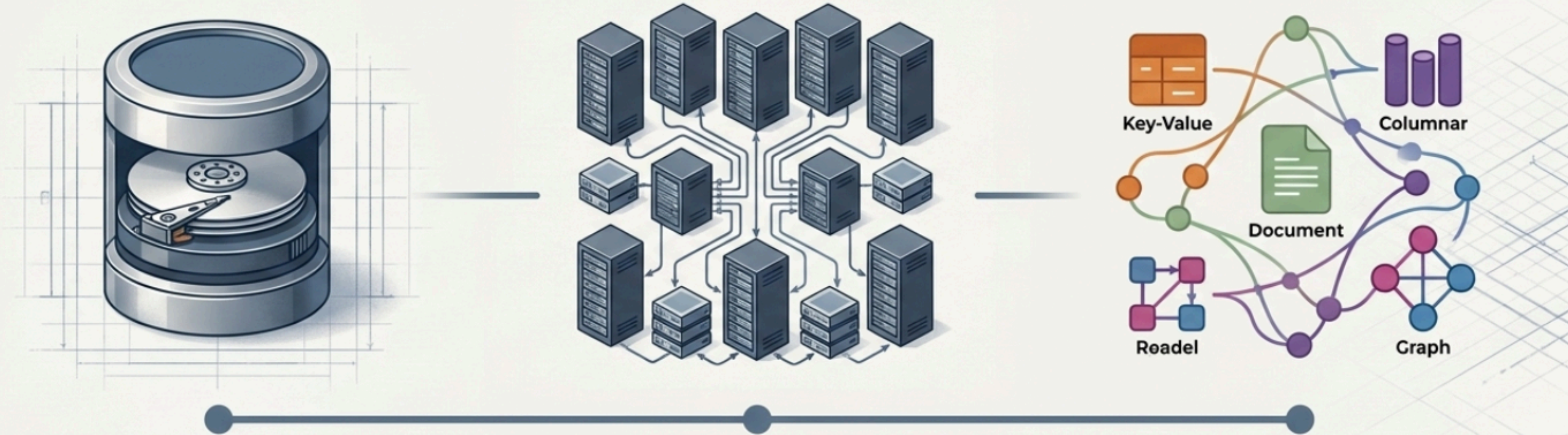


Facebook tiene que responder en menos de un segundo:
¿Qué amigos tuyos le dieron like a esta publicación?

¿Alguien se imagina esta consulta escrita como un JOIN en una tabla de amistades de billones de filas?



La evolución del problema de los datos



Paso 1: El Acceso (Ayudantía 1)

Un solo disco.
Por qué los índices importan.

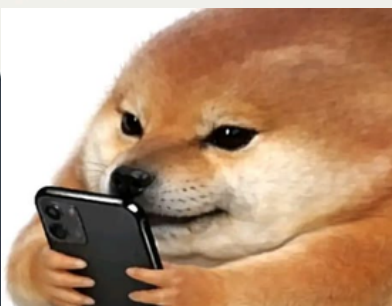
Paso 2: El Almacenamiento (Ayudantía 2)

Muchas máquinas (HDFS/Hadoop).
Cómo repartir el peso (Google GFS, 2004).

Paso 3: El Modelo (Hoy)

Si los datos ya están repartidos en
cientos de servidores... ¿cómo los
modelamos y consultamos?

NoSQL nace en la industria (Google, Amazon, Web Social) para resolver problemas que el modelo relacional físico ya no podía soportar.

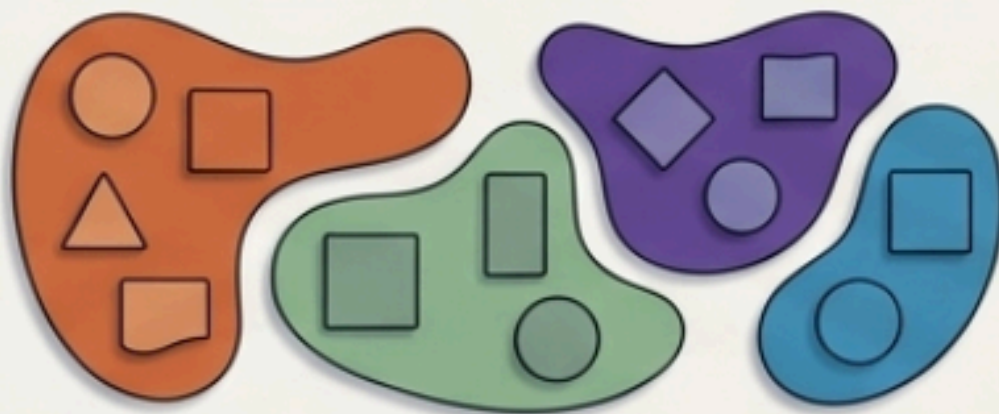


Las 3 reglas que cambian: Not Only SQL

El Esquema

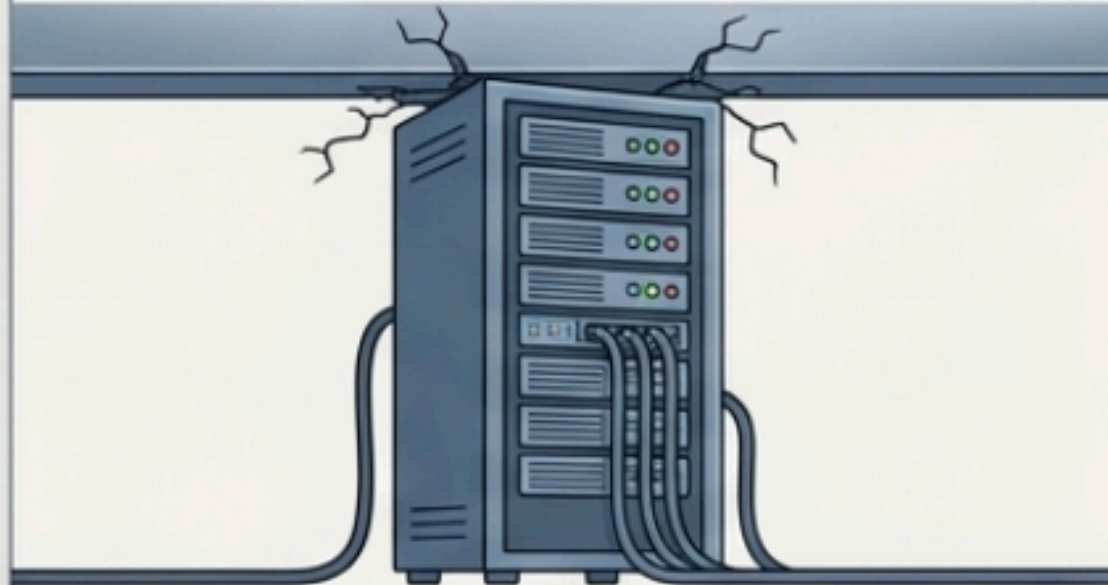


Relacional: Rígido, schema-on-write. Cambiar una columna es crítico.



NoSQL: Flexible, schema-on-read. Cada registro puede ser distinto.

El Escalamiento

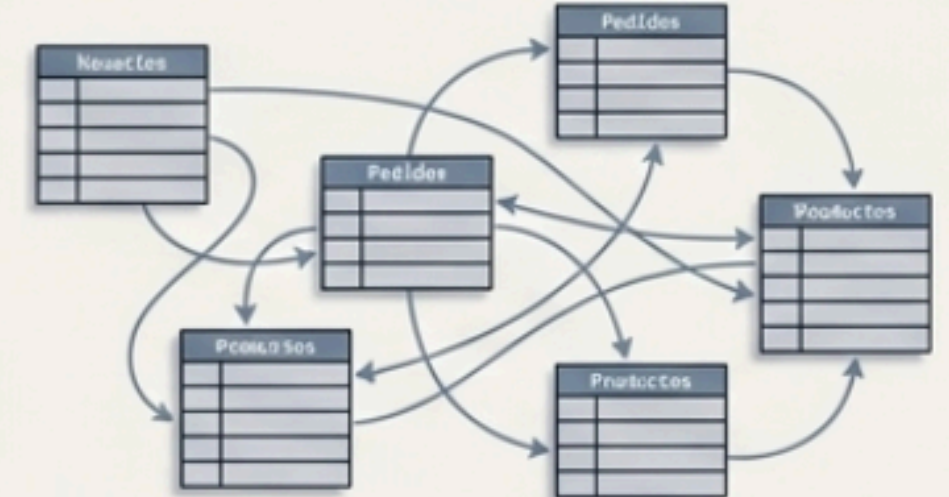


Relacional: Vertical. Tiene techo físico.



NoSQL: Horizontal. Se agregan máquinas, sharding automático.

Los JOINS



Relacional: Normalización para no repetir datos.

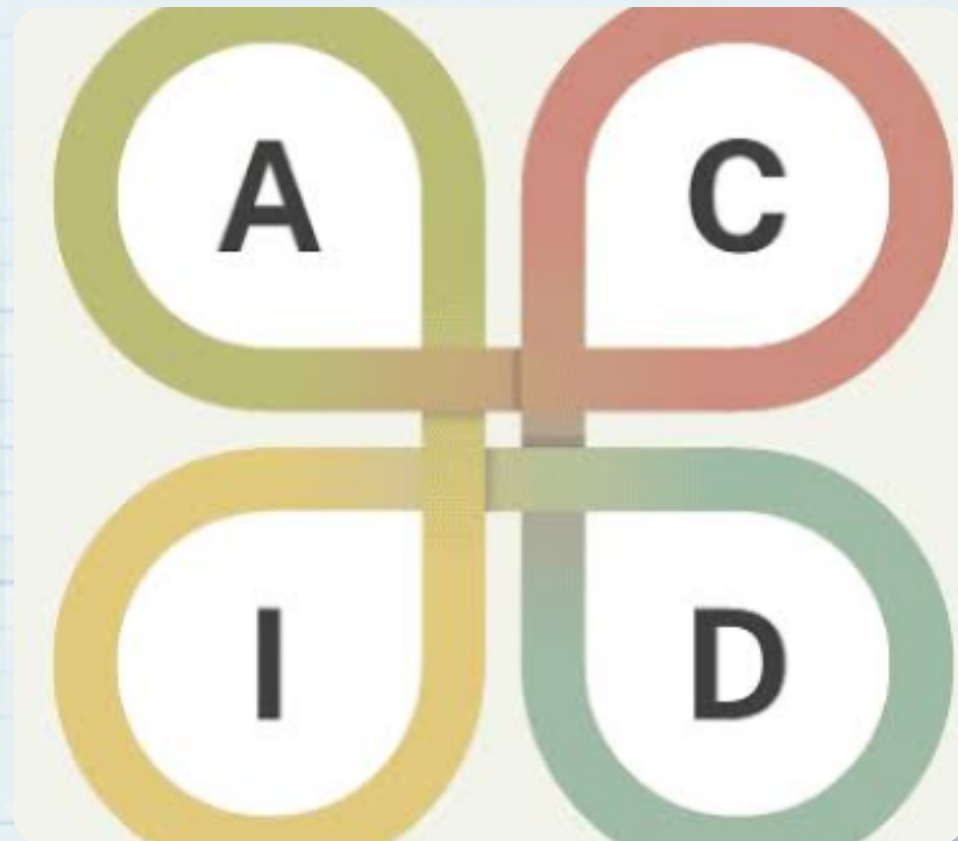


NoSQL: Desnormalización. Se guarda junto lo que se lee junto.

En SQL modelas según cómo se relacionan los datos. En NoSQL modelas según cómo vas a consultarlos.

Dos filosofías: ACID vs. BASE

El mundo Relacional



ACID

(Atomicidad, Consistencia, Aislamiento, Durabilidad)

La transacción pasa completa o no pasa nada. La base de datos nunca viola sus reglas. Estricto.

El mundo Distribuido



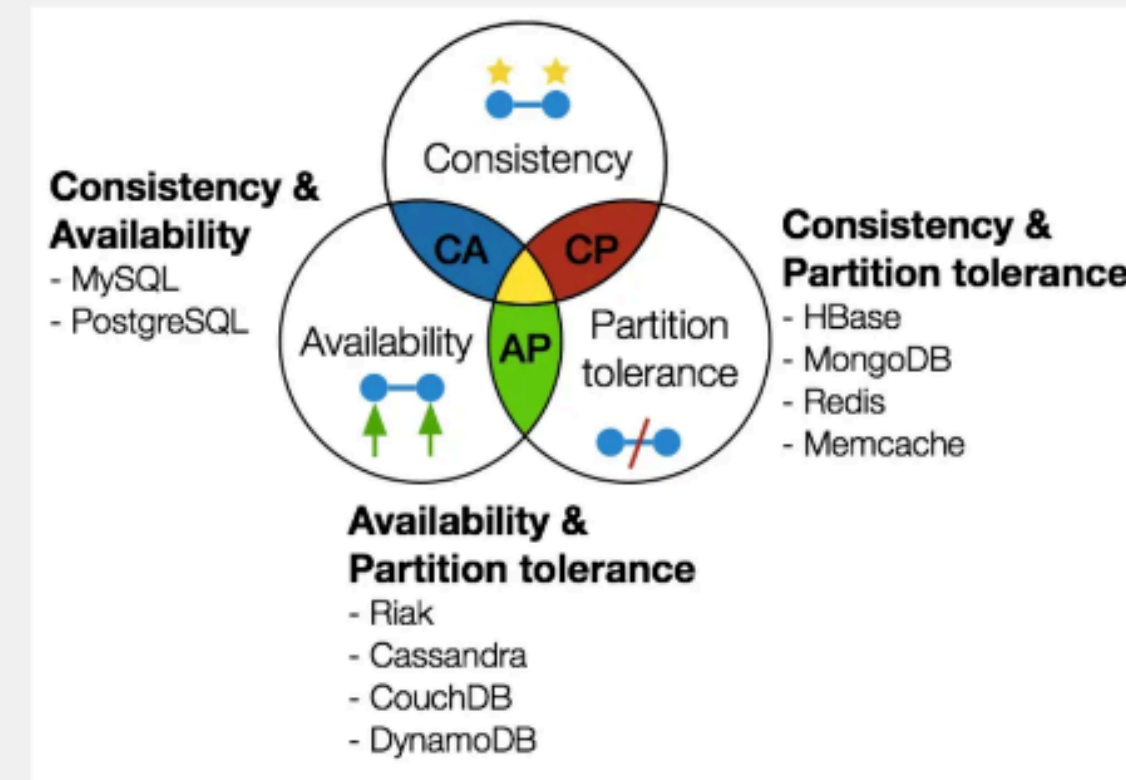
BASE

(Basically Available, Soft state, Eventual consistency)

El sistema responde siempre. Las réplicas pueden diferir por milisegundos, pero eventualmente convergen.

Teorema CAP

"En cualquier sistema de datos distribuido, es imposible garantizar simultáneamente la consistencia, la disponibilidad y la tolerancia a particiones. A la hora de diseñar el sistema, solo puedes elegir dos de estas tres propiedades."



Cuándo NO usar NoSQL

Datos Fuertemente Relacionales

Si necesitas transacciones estrictas y el volumen es manejable, PostgreSQL es imbatible.

Consultas Ad-Hoc

Si necesitas reportes arbitrarios que nadie previó al diseñar el sistema. SQL no obliga a modelar de antemano por consulta.

Equipos Pequeños

NoSQL traslada la complejidad (integridad, joins) desde la base de datos hacia el código de tu aplicación. Alguien tiene que mantener ese código.

Se justifica cuando el volumen, la velocidad o la forma de los datos rompen lo que una sola máquina relacional puede hacer. No por moda.

Un ecosistema, cuatro familias

Modelando la misma Red Social Universitaria según la pregunta que queremos responder.

Clave-Valor



```
SET sesion:8f3a91 {usuario:1024}
```

Búsqueda por clave en microsegundos (Sesiones, carritos).

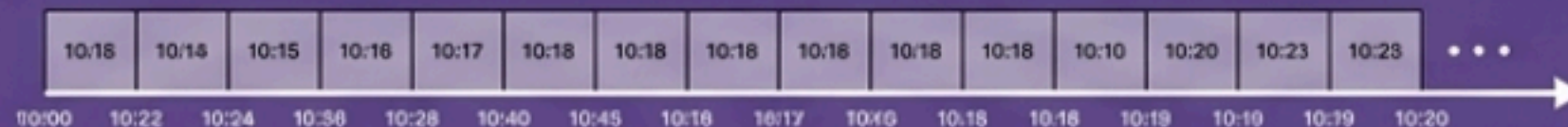
Documental

Un solo documento JSON con el perfil de Ana, sus arrays de amigos y publicaciones integrados.

Traer el objeto complejo completo en una sola lectura sin JOINS.



Columnar



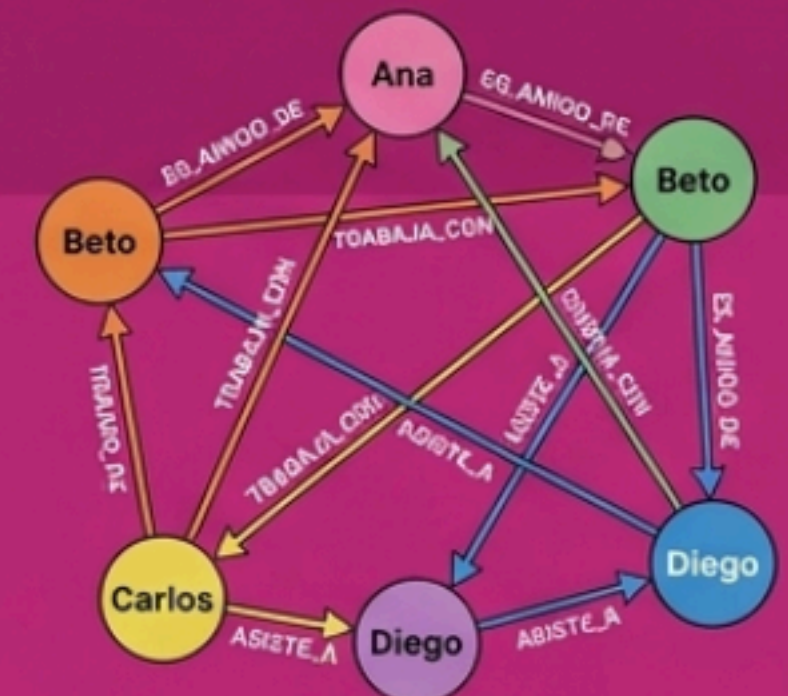
```
usuario_1024 | 10:15->login | 10:18->like
```

Escribir millones de eventos de telemetría y leer por rangos de tiempo.

Grafo

```
(Ana)-[:ES_AMIGO_DE]->(Beto)
```

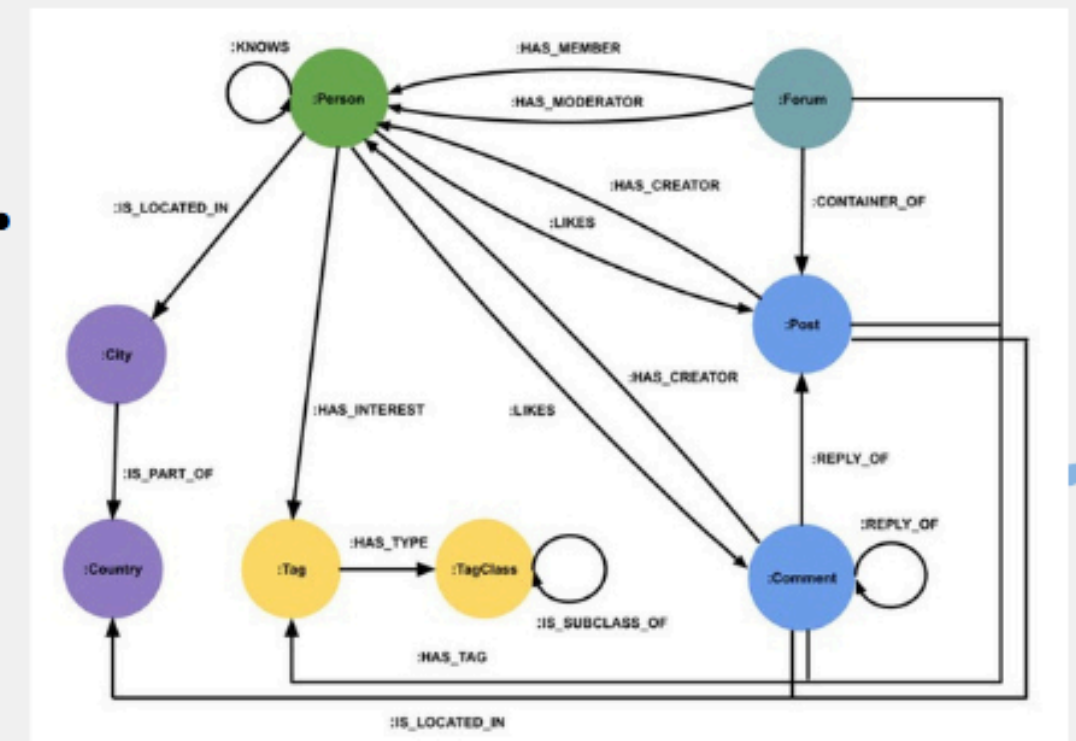
Recorrer relaciones complejas sin calcular JOINS.



Tipos de BD NoSQL

Graph databases

- Almacena los datos en nodos.
- Los nodos representan entidades.
- Las aristas que unen los nodos establecen las relaciones entre los nodos.
- Idealmente para datos muy conectados.
- Se usa mucho en redes sociales.
- Ejemplos: Neo4j, Amazon Neptune.



Tipos de BD NoSQL

Key-Value stores:

- **Almacena los datos en un diccionario no estructurado.**
- **Acceso rapido, Key es la llave de búsqueda.**
- **No permite consultas complejas.**
- **Ejemplos: Redis, DynamoDB, etc.**

Ventaja: Rendimiento demasiado alto.

Desventaja: Búsquedas limitadas (solo por Key).

Tipos de BD NoSQL

Document Databases

- Almacena datos en “documentos”.
- Los “Documentos” se guardan en formato JSON, BSON, XML.
- Las Colecciones guardan conjuntos de Documentos.



Tipos de BD NoSQL

Column-Oriented Databases

- Al contrario que las BD SQL estas almacenan los datos por columnas. Por lo tanto las filas guardan los atributos.
- Las columnas son el mismo tipo de dato.
- Ejemplos: Cassandra, HBase.

Row-oriented			
ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

Column-oriented					
Name	ID	Grade	ID	GPA	ID
John	001	Senior	001	4.00	001
Karen	002	Freshman	002	3.67	002
Bill	003	Junior	003	3.33	003

Tipos de BD NoSQL

Column-Oriented Databases

- Al contrario que las BD SQL estas almacenan los datos por columnas. Por lo tanto las filas guardan los atributos.
- Las columnas son el mismo tipo de dato.
- Ejemplos: Cassandra, HBase.

Row-oriented			
ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

Column-oriented					
Name	ID	Grade	ID	GPA	ID
John	001	Senior	001	4.00	001
Karen	002	Freshman	002	3.67	002
Bill	003	Junior	003	3.33	003

¿Cuándo usar cada BD NoSQL?

1 Documentos

Ejemplos: MongoDB, CouchDB

Catálogos de productos, CMS, perfiles de usuario: datos semi-estructurados que cambian de forma con frecuencia.

2 Clave-Valor

Ejemplos: Redis, DynamoDB

Cachés, sesiones de usuario, carritos de compra: lecturas y escrituras ultrarrápidas por una clave única.

3 Columnares

Ejemplos: Cassandra, HBase

IoT, series de tiempo, mensajería a gran escala: enormes volúmenes de escritura y consultas por columna.

4 Grafos

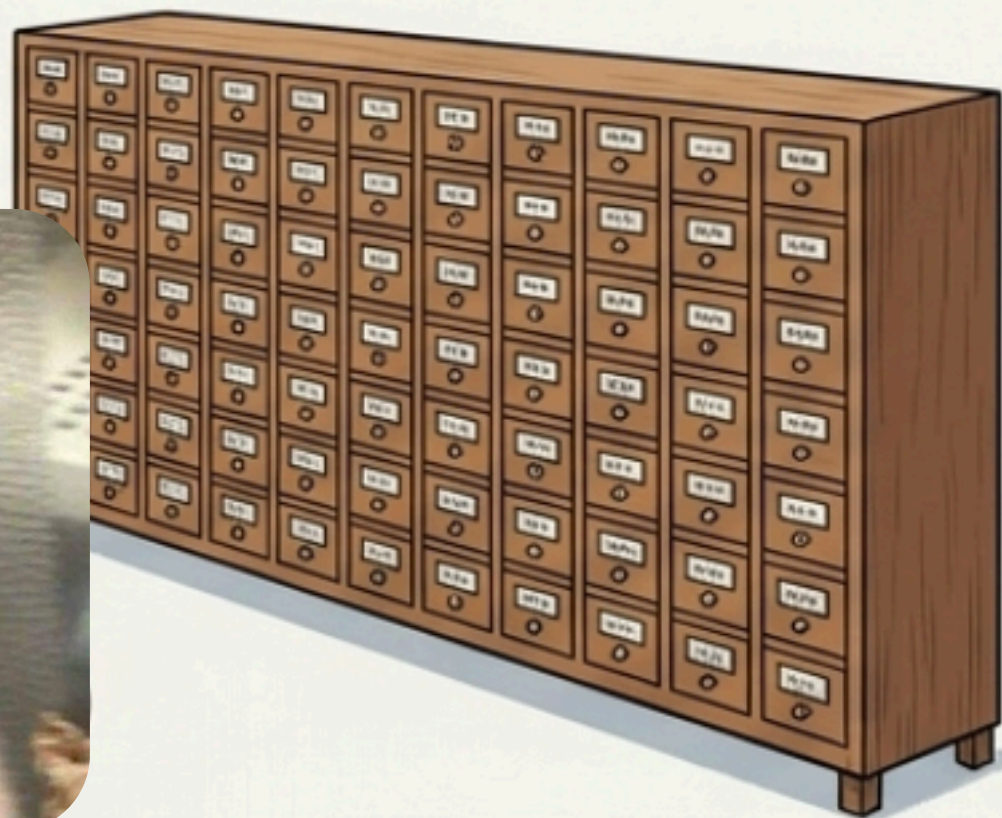
Ejemplos: Neo4j, ArangoDB

Redes sociales, motores de recomendación, detección de fraude: datos con relaciones muy interconectadas.

El secreto del Grafo: Index-Free Adjacency

¿Por qué encontrar amigos de amigos es instantáneo en un grafo y lentísimo en SQL?

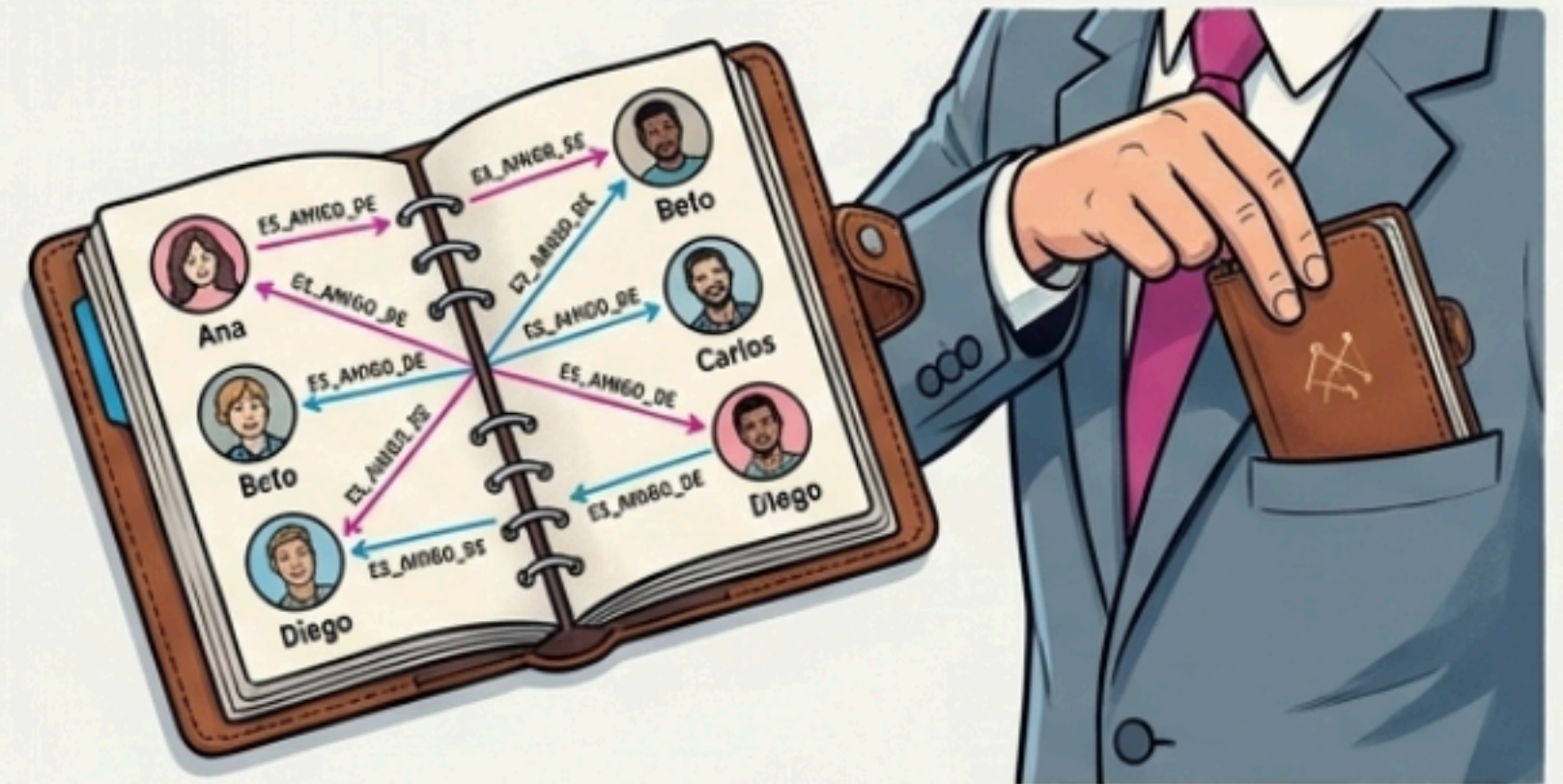
SQL: El Índice Global



En SQL, la relación se calcula. Cada salto requiere buscar en el **índice global** completo.

El **costo** depende del tamaño total de la base de datos.

Grafo: El Bolsillo



En un Grafo, la relación es un **puntero físico** guardado en el **nodo mismo**. Vas directo al vecino.

El **costo** depende solo de cuántos amigos tienes, no de si el mundo tiene mil o mil millones de personas.

CONTACTO

Gmail

- roberto.ibaceta@mail.udp.cl

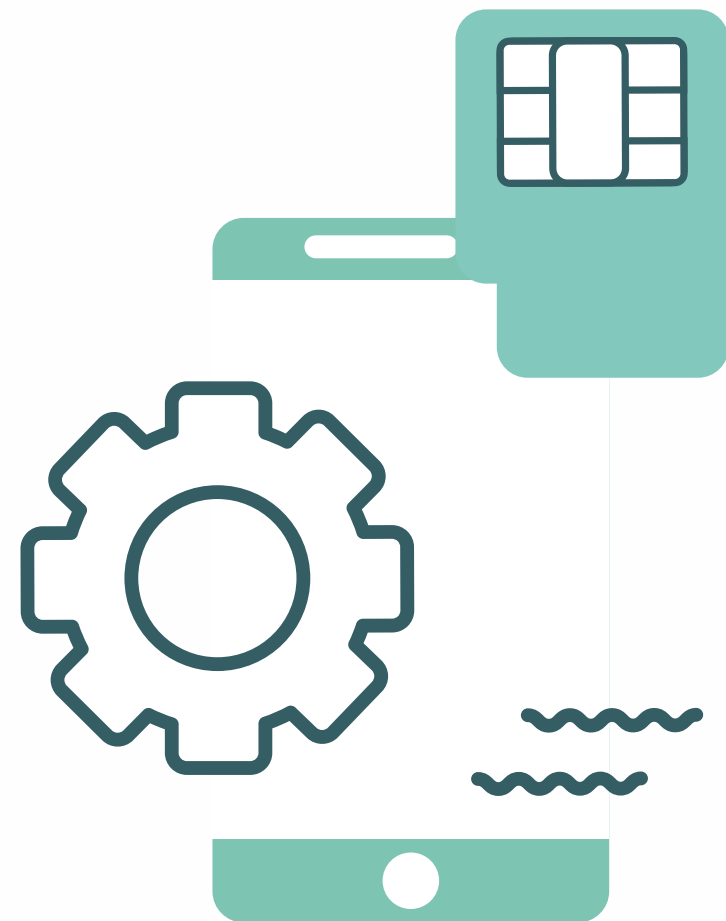
Wsp

- [+569 42663436](https://wa.me/56942663436)

Ds

- [roberibaz](#)





**MUCHAS
GRACIAS**

